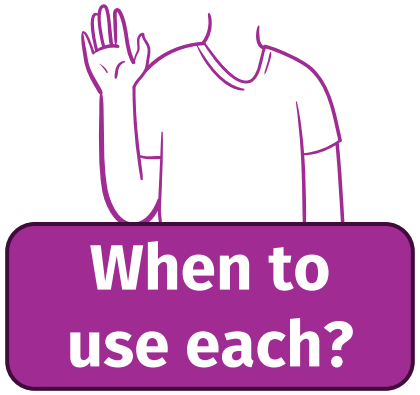
SENG 201
Software Engineering Foundations

[Week 3, Wednesday]



while / do while / for



While

- Unbounded iteration

```
while (!isTaskFinished()) {
    // do more work
}
```

Do While

- Guarantee of one iteration

```
do {
    // start or resume task
} while (!isTaskFinished());
```

For

- Bounded iteration
- Indexed iteration

```
for (int step = 0; step < nbSteps(); step++) {
    // do next step
}
```

For each

- Data structures
- Unindexed

```
for (Type item : dataStructure) {
    // process item
}
```


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Jumping out of iterations

```
String framedContent = "+----------------------\n";
for (String line : lines) {
    if (line.startsWith("//")) {
        // ignore comments
        continue;
    }
    if (line.equals(">> END <<")) {
        break;
    }
    framedContent += "| " + line + "\n";
}
framedContent += "+----------------------";
```

Skip the rest of the current iteration

continue

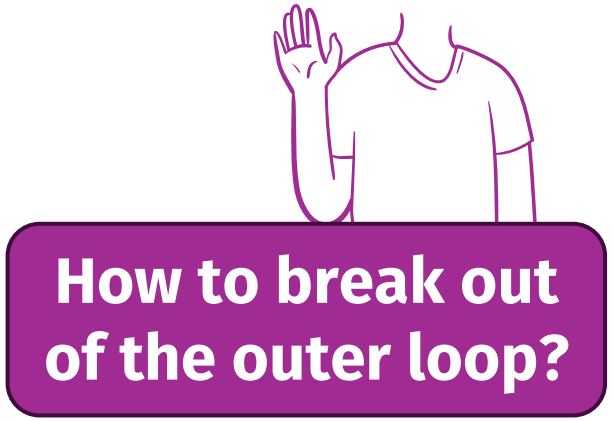
break

Skip the rest of the loop entirely

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Jumping out of iterations

```java
String[] lines = content.split("\n");
for (String line : lines) {
    // regex: split by whitespace
    String[] words = line.split("\\s+");
    for (String word : words) {
        if (word.equals("END_ALL")) {
            break; // end all processing
        }
        if (word.equals("//")) {
            continue; // to next line
        }
        System.out.println(word);
    }
    System.out.println("----");
}
```



© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

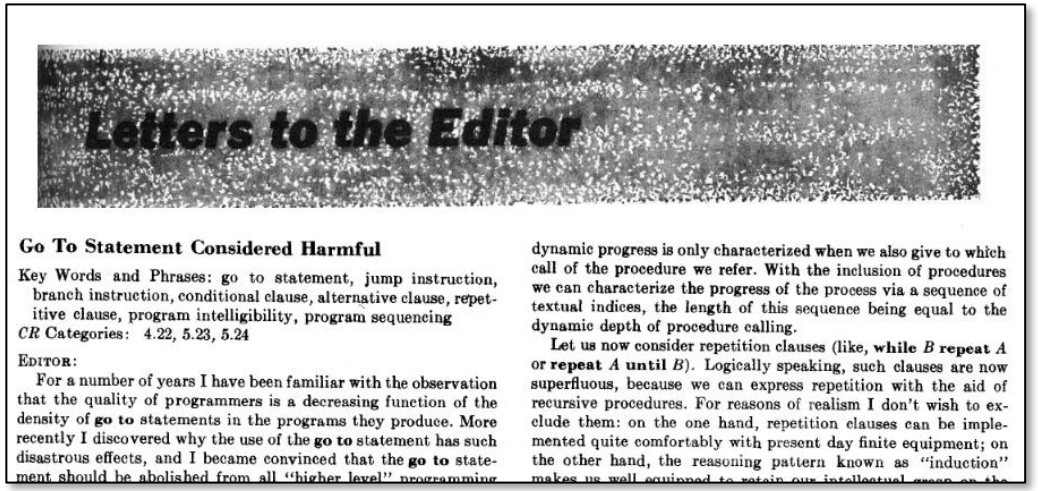
Jumping out of iterations

labels

labels

```java
String[] lines = content.split("\n");
outer: for (String line : lines) {
    // regex: split by whitespace
    String[] words = line.split("\\s+");
    inner: for (String word : words) {
        if (word.equals("END_ALL")) {
            break outer; // end all processing
        }
        if (word.equals("//")) {
            continue outer; // to next line
        }
        System.out.println(word);
    }
    System.out.println("----");
}
```

Don’t use too often!



Letters to the Editor

Go To Statement Considered Harmful

Key Words and Phrases: go to statement, jump instruction, branch instruction, conditional clause, alternative clause, repetitive clause, program intelligibility, program sequencing
CR Categories: 4.22, 5.23, 5.24

EDITOR:

For a number of years I have been familiar with the observation that the quality of programmers is a decreasing function of the density of **go to** statements in the programs they produce. More recently I discovered why the use of the **go to** statement has such disastrous effects, and I became convinced that the **go to** statement should be abolished from all "higher level" programming

dynamic progress is only characterized when we also give to which call of the procedure we refer. With the inclusion of procedures we can characterize the progress of the process via a sequence of textual indices, the length of this sequence being equal to the dynamic depth of procedure calling.

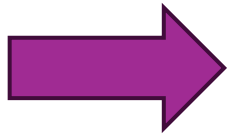
Let us now consider repetition clauses (like, **while** *B* **repeat** *A* or **repeat** *A* **until** *B*). Logically speaking, such clauses are now superfluous, because we can express repetition with the aid of recursive procedures. For reasons of realism I don't wish to exclude them: on the one hand, repetition clauses can be implemented quite comfortably with present day finite equipment; on the other hand, the reasoning pattern known as "induction" makes us well equipped to retain our intellectual grasp on the

Edsger W. Dijkstra, “Go To Statement Considered Harmful.” In Communication of the ACM, 11(3), 1968.

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Exercise: Finish the frame (right side)

```
+----------------------
| Very
| important
| information
+----------------------
```



```
+-------------+
| Very        |
| important   |
| information |
+-------------+
```


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Arrays



- → Constraints
- → Creation
- → Multi-dimensional arrays

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Arrays in Java

- Hold a fixed number of values (can’t be resized)
- All values have the same type
- Zero-indexed (first element at index 0)
  - Access using []

```
someArray[0]
```


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Declaration, initialization, creation

Declaration

```
String[] strings;
int[] ints;
boolean[] booleans;
```

Creation

```
strings = new String[10];
ints = new int[3];
booleans = new boolean[5];
```

Initialize values to 0/false/null

Declaration + initialization

```
String[] strings = { "a", "b", "c" };
int[] ints = { 1, 2, 3 };
boolean[] booleans = { true, false };
```

Error!

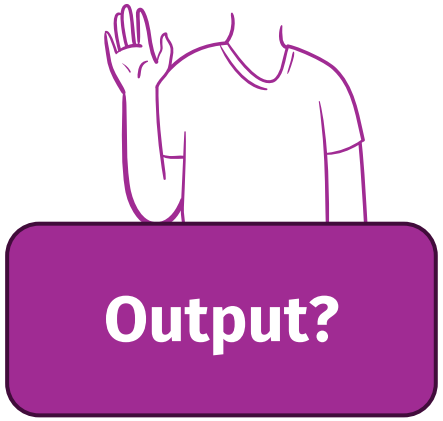
```
strings = { "a", "b", "c" };
ints = { 1, 2, 3 };
booleans = { true, false };
```

Creation + initialization

```
strings = new String[]{ "a", "b", "c" };
ints = new int[]{ 1, 2, 3 };
booleans = new boolean[]{ true, false };
```


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

```java
String[] original = { "a", "b", "c" };
String[] newValues = original;
newValues[0] = "d";
newValues[1] = "e";
original[0] = "f";
System.out.println(Arrays.toString(newValues));
```



© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Multi-dimensional arrays

- An array is declared by `type[]`
- What if `type` is an array?

`String[][]`

- Arrays have a fixed size
- We can change the elements of an array
- What if we reassign internal arrays?

Multi-dimensional arrays are just arrays that contain arrays

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Multi-dimensional arrays

```
String[][] array2d = new String[5][3];
array2d[0][0] = "Hello!";
array2d[2][1] = "middle";
```

```
int[][] points = { {0, 1}, {0, 2}, {1, 1}, {1, 2}};
points[3] = null;
```

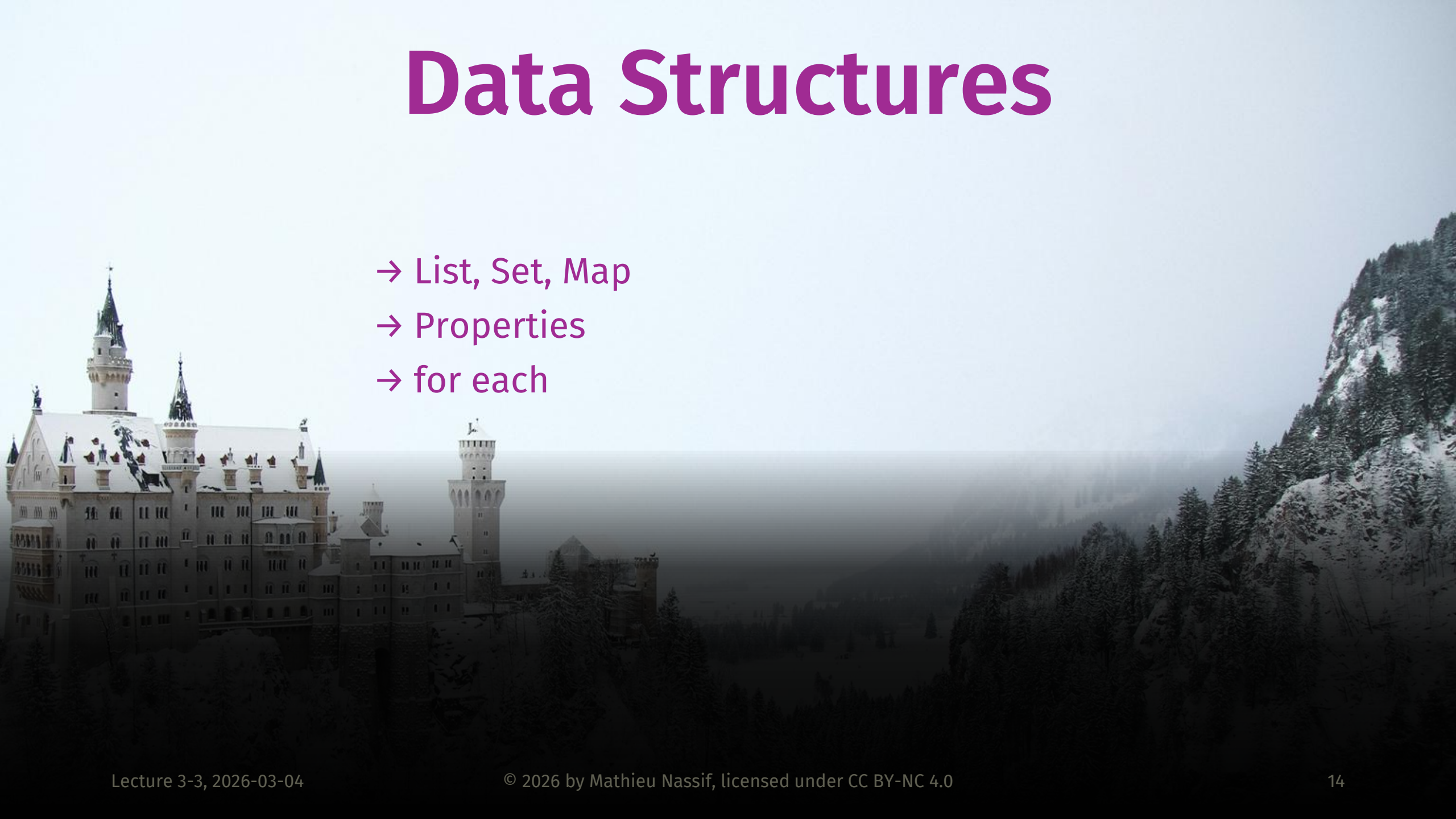
```
array2d[1] = new String[1];
array2d[4] = array2d[1];
array2d[1][0] = "single";
```



```
System.out.println(Arrays.deepToString(array2d));
```


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Data Structures



→ List, Set, Map

→ Properties

→ for each

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Java Collections API

Data structures available as part of the standard library (java.util package)

What we've encountered so far:

- String
- Object
- Arrays
- System
- List
- ArrayList

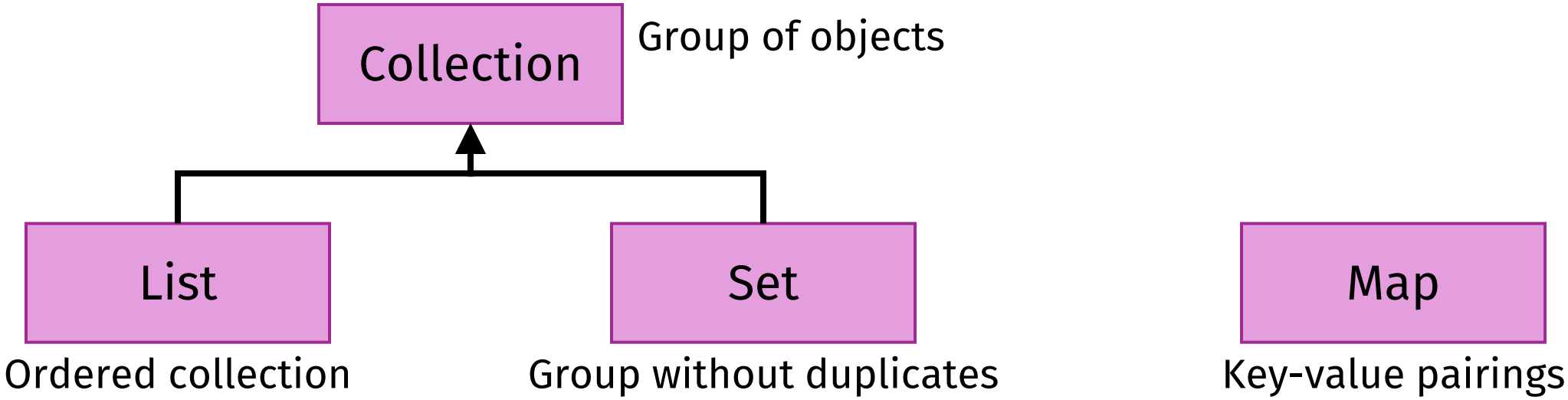
New types we'll look at:

- Collection
- List
- ArrayList
- LinkedList
- Set
- HashSet
- Map
- HashMap
- Entry
- EntrySet

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

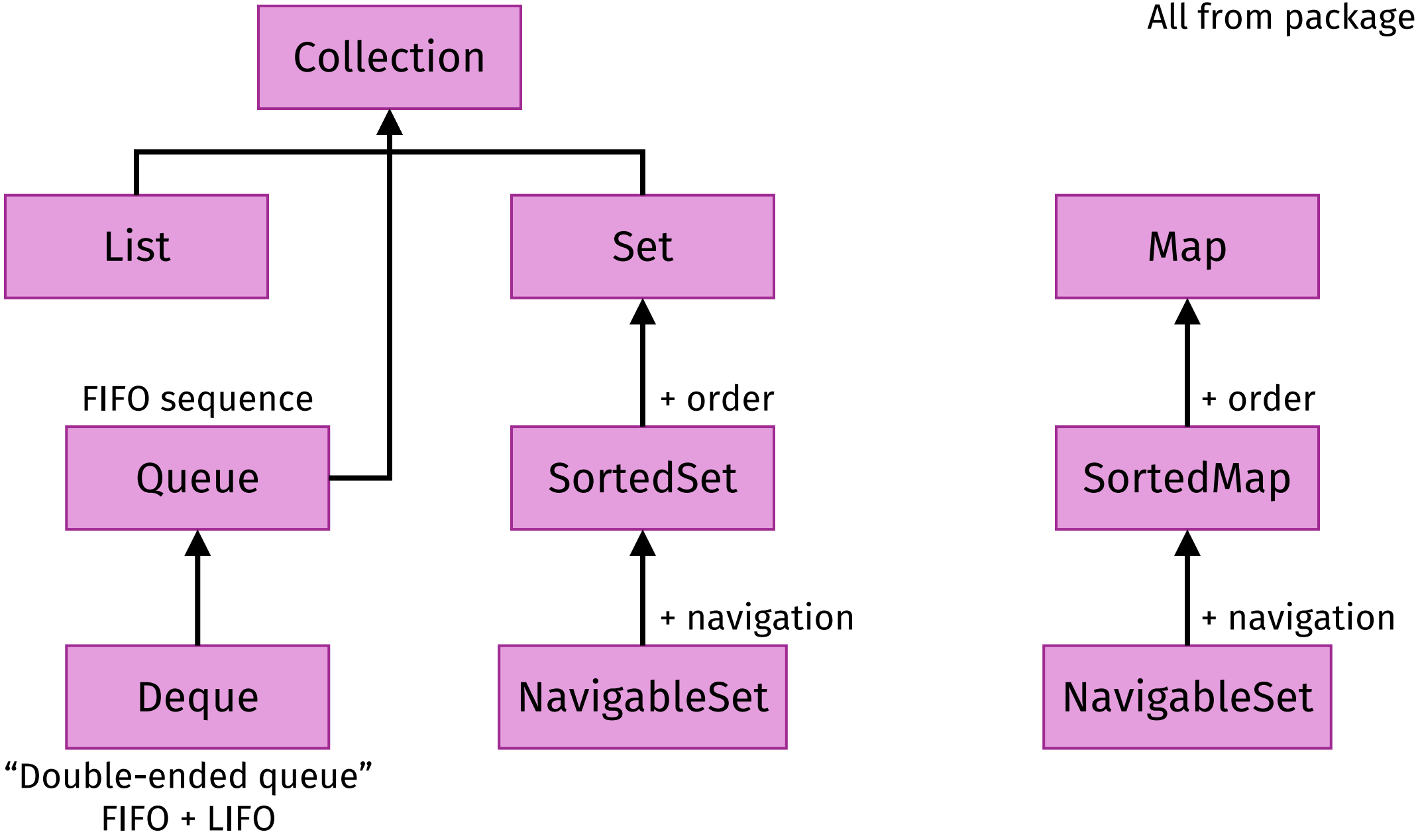
All from package java.util

Java Collections API



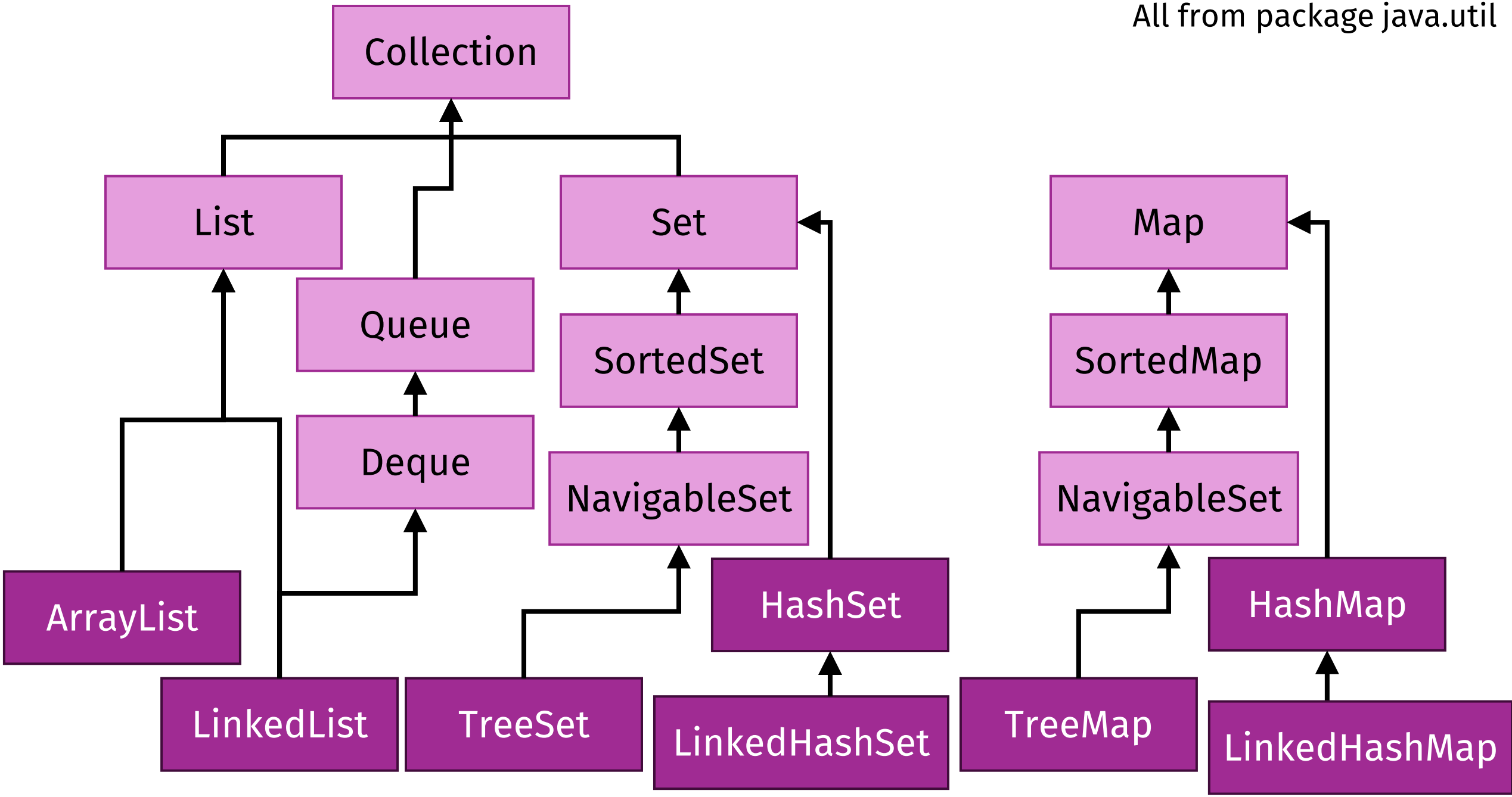
© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

All from package java.util



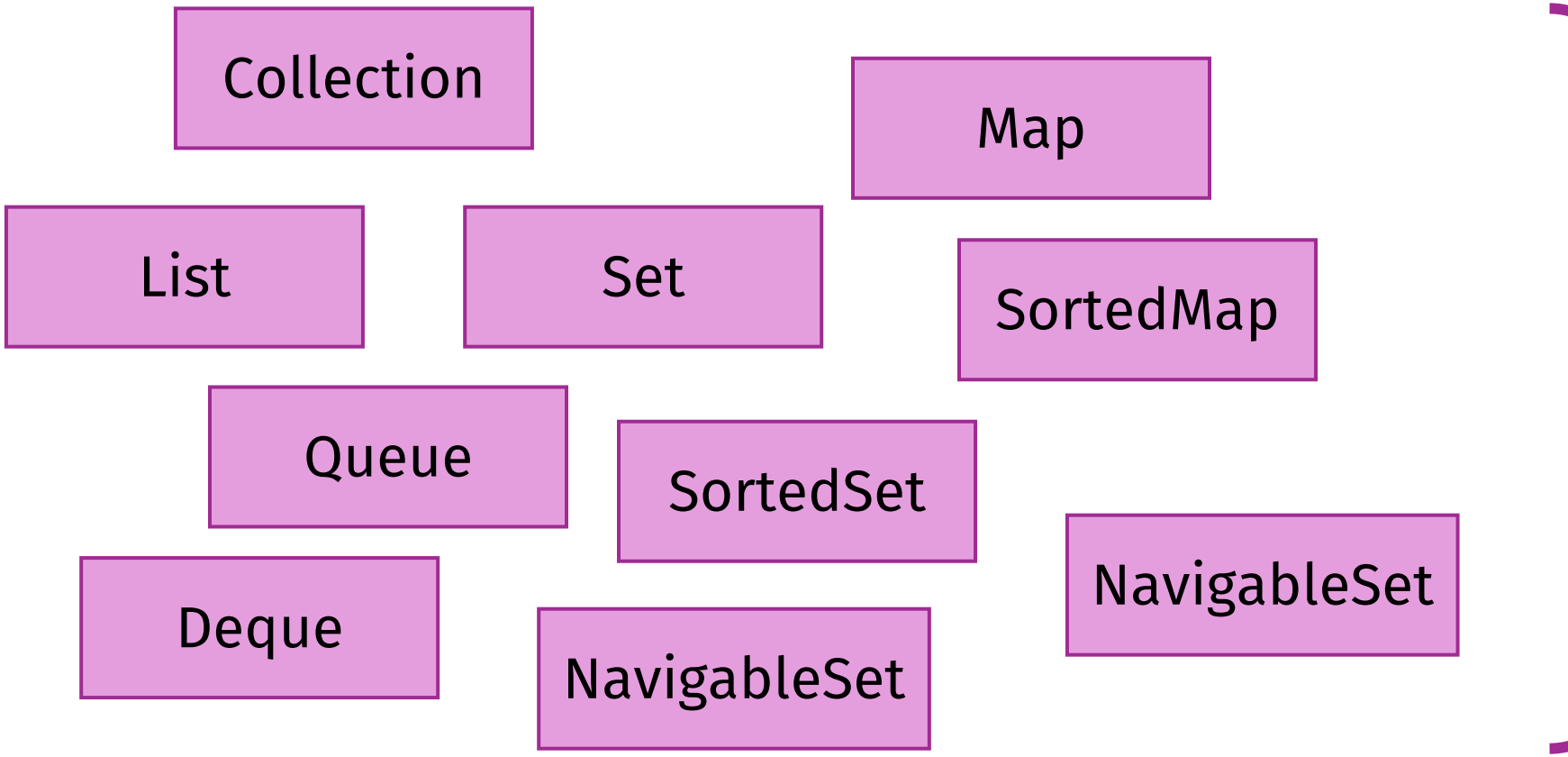
© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

All from package java.util

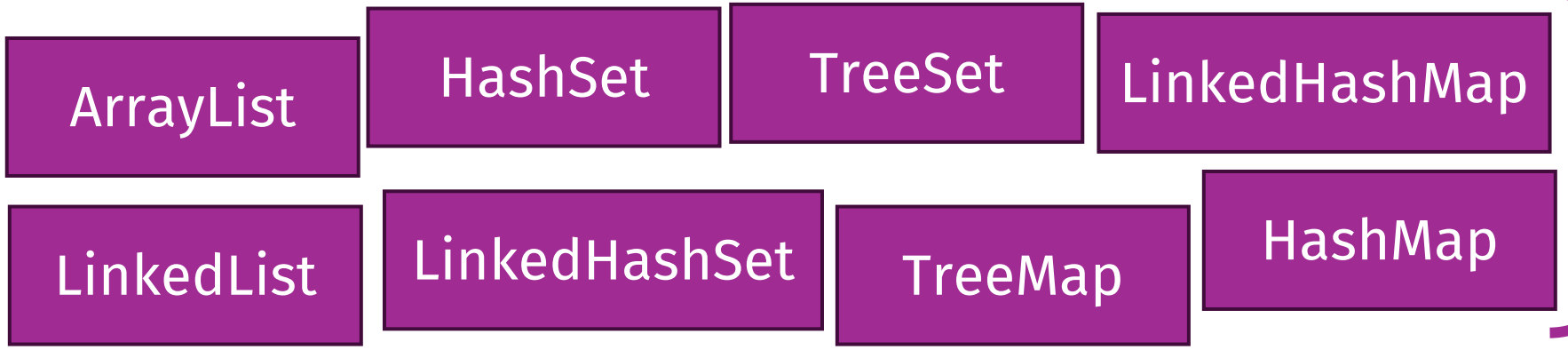


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

All from package java.util

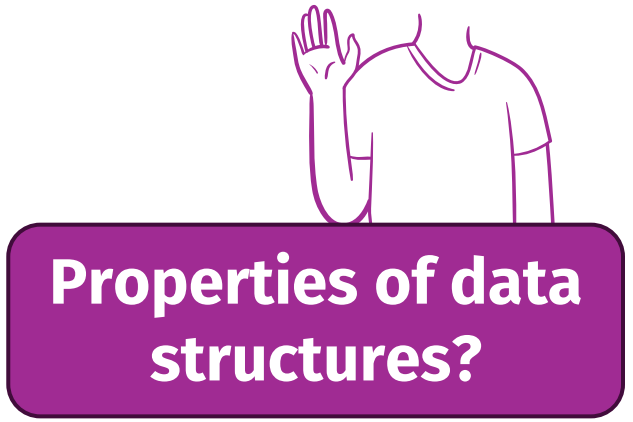


Interfaces:
use as types,
but can't be created
with a constructor



Classes:
use as types and as
actual instances

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0



What do you actually need to know?

- Be able to choose the right type for each context

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Properties of data structures

- **Duplicates** allowed
- **Null** allowed
- **Ordered**: fixed iteration order
- **Sorted**: fixed iteration order based on elements (e.g., ascending)
- **Mutable**: can replace elements
- **Resizable**: can add and remove elements
- **Indexed**: can access elements at any specific index
- **Navigable**: can access element before or after current one
- **Synchronized**: can be shared across threads

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Properties of data structures

	Nulls	Duplicates	Ordered	Sorted	Mutable	Resizable	Indexed
Java arrays	YES	YES	YES	NO	YES	NO	YES
List	?	YES	YES	NO	?	?	YES
ArrayList	YES	YES	YES	NO	YES	YES	YES
LinkedList	YES	YES	YES	NO	YES	YES	YES
Set	?	NO	?	?	?	?	NO
HashSet	YES	NO	NO	NO	YES	YES	NO
LinkedHashSet	YES	NO	YES	NO	YES	YES	NO
TreeSet	YES	NO	YES	YES	YES	YES	NO
...							

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

How to use them

```java
List<String> list = new ArrayList<>();
list.add("a");      // [a]
list.add("b");      // [a, b]
list.add("c");      // [a, b, c]
list.add("d");      // [a, b, c, d]
list.remove(1);     // [a, c, d]
list.remove("c");   // [a, d]
String d = list.get(1);
int size = list.size();
System.out.println(d);
System.out.println(size);
```

```java
Set<String> set = new HashSet<>();
set.add("a");           // [a]
set.add("b");           // [a, b]
set.add("d");           // [a, b, d]
set.add("d");           // [a, b, d]
set.addAll(list);       // [a, b, d, c]
set.remove("b");        // [a, d, c]
set.removeAll(list);    // [d]
// String d = set.get(0);   // NOT POSSIBLE
int setSize = set.size();
System.out.println(setSize);
```

```java
Map<String, Integer> map = new HashMap<>();
map.put("one", 1);      // [one=1]
map.put("two", 2);      // [one=1, two=2]
map.put("three", 3);    // [one=1, two=2, three=3]
map.put("one", 3);      // [one=3, two=2, three=3]
map.remove("one");      // [two=2, three=3]
int two = map.get("two");
int mapSize = map.size();
System.out.println(two);
System.out.println(mapSize);
```


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

How to use them

- List<String>, Map<String, Integer>
  - *Generic* types: <type argument> indicates the type of elements

- Cannot contain primitive values
  - Must use wrapper classes (Integer, Double, etc.)
  - Rely on auto-boxing and auto-unboxing

- Common methods
  - add()/put(), remove(), get(), contains(), size(), clear(), isEmpty()

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

How to use them

- Iteration

```
List<String> list = new ArrayList<>();
list.add("a");      // [a]
list.add("b");      // [a, b]
list.add("c");      // [a, b, c]
list.add("c");      // [a, b, c, c]
for (String element : list) {
    System.out.println(element);
}
```

```
Set<String> set = new HashSet<>();
set.add("a");           // [a]
set.add("b");           // [a, b]
set.add("d");           // [a, b, d]
set.add("d");           // [a, b, d]
set.addAll(list);       // [a, b, d, c]
for (String element : set) {
    System.out.println(element);
}
```



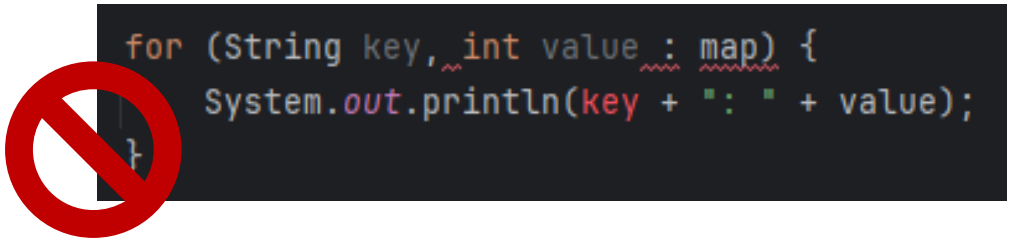
Do not modify the data structure inside the loop!

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

How to use them

- Iteration

```java
Map<String, Integer> map = new HashMap<>();
map.put("one", 1);      // [one=1]
map.put("two", 2);      // [one=1, two=2]
map.put("three", 3);    // [one=1, two=2, three=3]
```



```java
for (String key, int value : map) {
    System.out.println(key + ": " + value);
}
```

```java
for (String key : map.keySet()) {
    System.out.println(key + ": " + map.get(key));
}

// map.values() is a Collection<Integer>
// Can't be a set (allow duplicates) or a list (unordered)
for (int value : map.values()) {
    System.out.println(value);
}

for (Entry<String, Integer> pair : map.entrySet()) {
    System.out.println(pair.getKey() + ": " + pair.getValue());
}
```


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Even more options

`List`, `Set`, *and* `Map` *provide static methods to create collections.*

```
List<Integer> intList = List.of(1, 2, 1);
List<Integer> listCopy = List.copyOf(intList);

Set<Integer> intSet = Set.of(1, 2);
Set<Integer> setCopy = Set.copyOf(intSet);

Map<Integer, String> intMap = Map.of(1, "one", 2, "two");
Map<Integer, String> mapCopy = Map.copyOf(intMap);
```


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Even more options

	Nulls	Duplicates	Ordered	Sorted	Mutable	Resizable	Indexed
array	YES	YES	YES	NO	YES	NO	YES
ArrayList	YES	YES	YES	NO	YES	YES	YES
List.of(...)	NO	YES	YES	NO	NO	NO	YES
HashSet	YES	NO	NO	NO	YES	YES	NO
TreeSet	YES	NO	YES	YES	YES	YES	NO
Set.of(...)	NO	NO	NO	NO	NO	NO	NO
...							

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

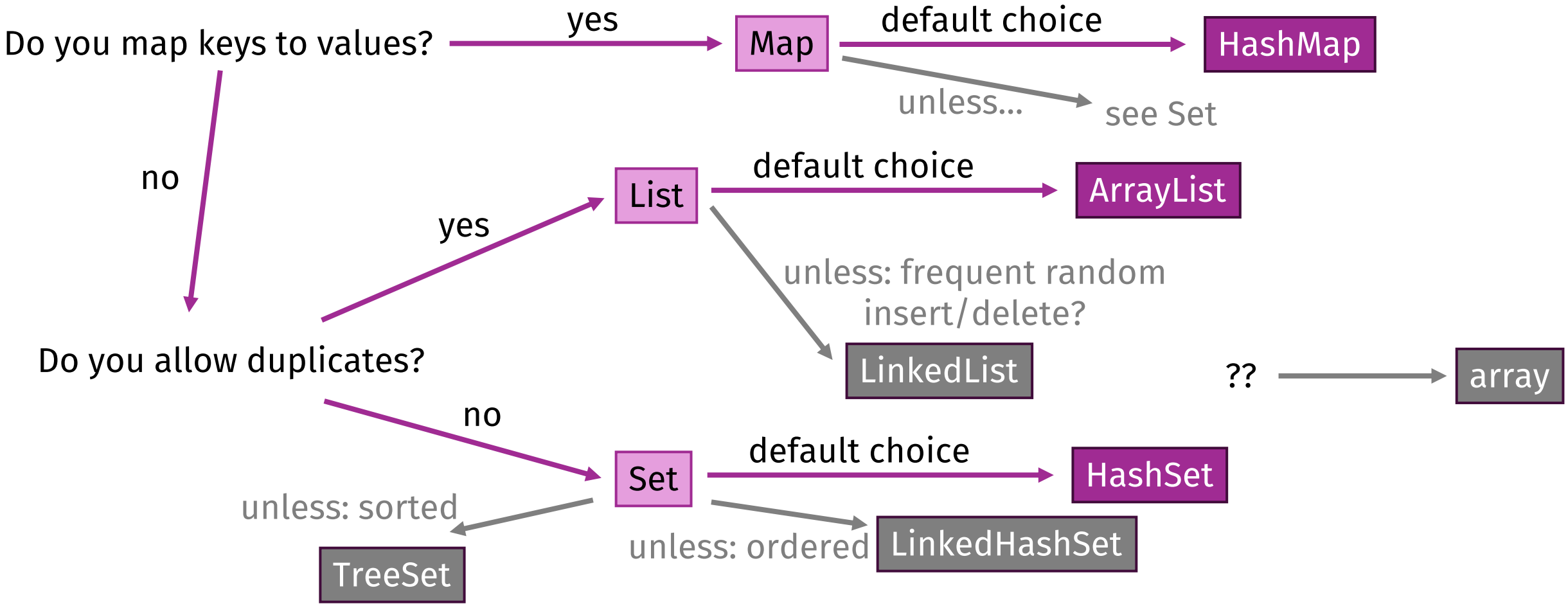
Multi-dimensional data structures

```
Map<String, List<List<Integer>>> factors = new HashMap<>();
factors.put("twelve", List.of(
        List.of(12),
        List.of(2, 6),
        List.of(2, 2, 3),
        List.of(3, 4)
));
boolean contain12 = factors.get("twelve") // list of lists for 12
        .get(0) // first list of factors
        .contains(12);
```


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

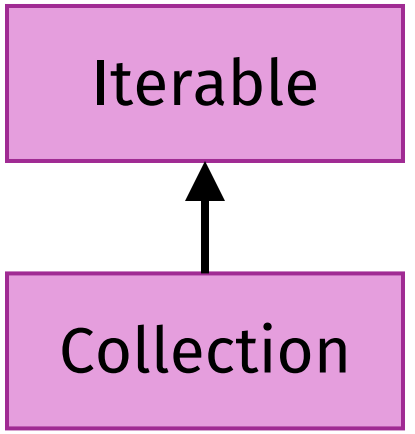
Choosing a data structure

Purple: should know first
Grey: also good to know
*There are even more to learn, eventually



© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Creating your own data structure



- Implement any interface or extend any class from the Collections API to create custom data structures.

- Implement Iterable interface to use a class in a for-each loop.

```
public class Notebook implements Iterable<Note> {

    @Override
    public Iterator<Note> iterator() {
        return notes.iterator();
    }
```

```
for (Note note : notebook) {
    System.out.println(note.content());
}
```


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

RECAP

→ break, continue

→ Arrays

→ Collections API



© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0